

Gaps, Optimizations & Feature Roadmap

Project: Single-pass drone/handheld video → georeferenced, walkable 3D model **Status date:** 2026-09-24

Legend: `wire` = capability already exists in code, just not connected · `build` = new work · `S/M/L` = effort ·

`△dep` = needs a new package or model weights

This document lists what exists today (verified against the codebase), what is missing, and what to build, organised by capability. It is deliberately honest: several requested features **already exist but are orphaned** (test-only or CLI-only), so the cheapest wins are wiring, not new algorithms.

A. Measurements & analysis

The real engine `scripts/survey_measure.py` (1688 lines) already implements distance, polygon/true/projected area, height-above-ground, volume & cut/fill, slope/aspect, cloud snapping, an RSS uncertainty budget, and GeoJSON/CSV export — but it is **test-only** (no HTTP/CLI entry point). The shipped product exposes only four naive viewer-pick kinds (point/distance/height/area) with no snapping or uncertainty.

ID	Item	Type	Effort
A1	Wire the real engine to HTTP + viewer (distance/area/volume/slope with snapping + uncertainty)	wire	M
A2	Volume / cut-fill tool (stockpiles, spoil, earthworks)	wire	M
A3	Per-class measurement (click a labelled building/road → footprint, height, façade area)	build	M
A4	Persist + export measurement sets as GeoJSON/CSV with CRS + uncertainty	wire	S
A5	Fix trust: walktest reports 9.6 m but only 4.2 m travelled (distance-inflation); gate misses it	build	S

B. Furniture / interior placement

`scripts/build_objects.py` only **detects** existing furniture boxes. Placement is now built end-to-end in `scripts/workspace_place.py` (a new adapter mirroring `workspace_measure.py`) + a `placements.json` store + a viewer render/collide path + a “Place” tab.

ID	Item	Type	Effort	Status
B1	Object placement API: drop a real-scale item, snap to floor, orient, move, delete	build	L	done — <code>placements</code> create/update/delete routes; snap via the scene heightfield; yaw + scale
B2	Whole-house multi-placement + saved layouts	build	L	done — many items persist per scene; layout exports as GeoJSON
B3	Furniture library (built-in primitives + glTF import)	build	M	partial — 12 real-dimension primitives shipped; glTF model import not built
B4	Collision for placed items so walk/game mode respects them	build	M	done — placed boxes go on a compound static body the character’s rays respect
B5	Room detection (walls/floor/ceiling) → per-room area + clearance check	build	L	partial — per-drop footprint fit + wall-clearance + ceiling check against the coverage grid; full room segmentation not built
B6	Reliable metric scale for interiors (see E1) — prerequisite for all of B	build	M	unchanged — placement inherits scene scale; <code>room_w_json1</code> is metric via AR pose-prior

C. Gaming / first-person walkthrough

Ammo.js physics, a character controller, an A* route, and a full `combat.js` + triangle-graph `nav.js` NavMesh already exist. But `nav.json` is baked **manually** per scene (`tools/navbake`) and is missing for `room_w_json1` , so game mode fails there.

ID	Item	Type	Effort
C1	Bake navmesh inside the pipeline (auto <code>nav.json</code>) → game-ready in one run	wire	M
C2	First-person walk mode as a first-class viewer option	wire	S
C3	Teleport/waypoint + minimap for navigating a scanned building	build	M
C4	Game export: package a scene as a standalone playable (glTF + nav.json + spawn)	build	L
C5	Multiplayer/agent sim hooks (bots exist) surfaced as a feature	wire	M

D. Reconstruction quality (the six key challenges)

ID	Challenge	Item	Type	Effort
D1	Dynamic objects	<code>survey_dynamics.py</code> (background model, motion mask, epipolar inconsistency) is implemented but never executed in the main pipeline — wire it	wire	M
D2	Textureless / few-view / blur	Monocular metric-depth prior (Depth-Anything/Metric3D) to regularise COLMAP	build	L Δ dep
D3	Variable illumination/shadows	Photometric normalisation pass (<code>survey_photometry</code> exists — check wiring)	wire	M
D4	GPS inaccuracy / sensor noise	GNSS uncertainty + clock-offset handling exists in survey lane — surface + default-on	wire	S
D5	Limited viewing angles	Honest coverage/occlusion map (<code>survey_visibility</code> / <code>survey_occlusion</code>) as a viewer layer	wire	S
D6	(quality)	Learned segmentation to upgrade heuristic semantics → real classes	build	M Δ dep

E. Scenario-specific optimizations

ID	Item	Type	Effort
E1	<code>room_w_json1</code> : currently <code>character_height 0.15</code> (diorama scale) + 7.6 m loop + distance-inflation → fix interior scale, re- validate walk	build	M
E2	Preset audit: re-tune room/drone/object/build ing presets against each sample scene; record which pass the gate	wire	M
E3	Small-object / close-up: object-preset sharpness + collider voxel tuning	wire	S
E4	Large aerial (<code>temple is partial</code>): canopy culling + coverage-gate robustness	wire	M

F. Georeferencing & accuracy

ID	Item	Type	Effort
F1	One-click survey path in- app (telemetry → align → CRS → checkpoint RMSE); create path is CLI-only today	wire	M
F2	GCP/checkpoint upload + auto accuracy report (turns “unverified” → “verified”)	build	M
F3	Vertical datum options (EGM geoid) so heights are MSL, not just ellipsoidal	build	M Δ dep

G. Export & deliverables

ID	Item	Type	Effort
G1	Textured-mesh UV bake (currently per-vertex colour only; <code>claims_textured_mesh:</code> <code>False</code>)	build	L
G2	Validate exports with real GDAL/laspy/PDAL (currently self-parsed)	wire	M Δ dep
G3	One-click project bundle (model + measurements + report + CRS) as ZIP	build	S

H. Performance / real-time

ID	Item	Type	Effort
H1	Progressive reconstruction (<code>survey_progressive</code>) is manual-flag only → stream a partial model as it builds	wire	M
H2	The <15-min gate is documented unreachable at current budget → profile + a fast path	build	L

I. Product / platform polish

ID	Item	Type	Effort
I1	Scene comparison / change-detection (two scans of the same place)	build	L
I2	Share links / read-only viewer for a scan	build	M
I3	Annotations with photos + report PDF export (inspection)	build	M
I4	Batch processing queue (many scenes)	build	M

Application coverage (can a customer use it today?)

Application	Usable now?	Blocking gaps
Rapid mapping / situational awareness	Mostly	D1 dynamics, H1 progressive
Infrastructure inspection	Partial	A1 real measurements, A5 trust, I3 report
Survey / construction (volumes, as-built)	Partial	A1/A2, F1/F2, G2
Interior design / furniture placement	Yes (primitives)	B3 glTF import, B5 full room segmentation
Gaming / FVP walkthrough	Partial (needs manual nav bake)	C1–C4
Disaster response	Partial	D1, D5 coverage, A2 volume
Heritage / virtual tour	Mostly	C2 walk, I2 share

Recommended build order

1. **A1–A5** — measurement (huge, already built, just orphaned)
2. **C1** — auto-navmesh → unlocks walk + FVP games for every scene
3. **D1** — dynamic-object masking (already built, never run)
4. **E1 / B6** — fix interior scale → then **B1–B2** furniture placement
5. **D2 / D6** — learned depth + segmentation (needs weights; confirm before any download)

This turns existing-but-dead code into shipped capability first, then builds the genuinely new features.